How to write a paper

Paper = a thing with expected parts

Here’s my paper template (may not work for you)

http: //tiny.cc/ advice22

The take away

(A) remove biased labels; and (B) rebalance internal distributions such that they are equal based on class and sensitive attributes

Related work: respect and disrespect the past

Title,abstract: Important! This is how you attract reviewers.

1. Many people Do something

2. There is a problem

3. Because of some novel/unique perspective, we might have a fix (which no one else has tried before)

4. What we did

Research questions (very quick summary of results)

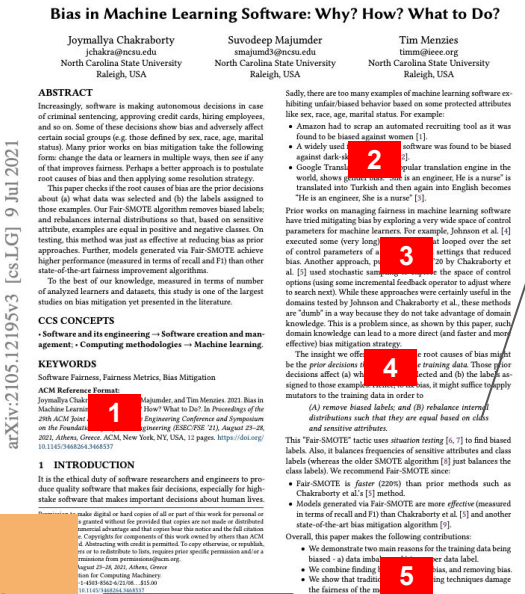
5. List of contributions

6. CYA: Caveats, limitations, expectation management,

Page2 groovy graphic

The throw
In summary, prior work suffered from: (1) Some methods only find bias, without trying to fix it; (2) Some methods for fixing bias have an undesired side effect: leaner performance was degraded.

For the rest of this paper, we explore a solution that finds root causes of bias, and directly implement mitigation of those causes (resulting in less bias and better performance than seen in prior work).



arXiv:2105.12195v3 [cs.LG] 9 Jul 2021

Bias in Machine Learning Software: Why? How? What to Do?

Joymallya Chakraborty, jchakra@ncsu.edu, North Carolina State University, Raleigh, USA
Suvodeep Majumder, smajumd3@ncsu.edu, North Carolina State University, Raleigh, USA
Tim Menzies, timm@ieee.org, North Carolina State University, Raleigh, USA

ABSTRACT

Increasingly, software is making autonomous decisions in case of criminal sentencing, approving credit cards, hiring employees, and so on. Some of these decisions show bias and adversely affect certain social groups (e.g. those defined by sex, race, age, marital status). Many prior works on bias mitigation take the following form: change the data or learners in multiple ways, then see if any of that improves fairness. Perhaps a better approach is to postulate root causes of bias and then applying some resolution strategy.

This paper checks if the root causes of bias are the prior decisions about (a) what data was selected and (b) the labels assigned to those examples. Our Fair-SMOTE algorithm removes biased labels; and rebalances internal distributions so that, based on sensitive attribute, examples are equal in positive and negative classes. On testing, this method was just as effective at reducing bias as prior approaches. Further, models generated via Fair-SMOTE achieve higher performance (measured in terms of recall and F1) than other state-of-the-art fairness improvement algorithms.

To the best of our knowledge, measured in terms of number of analyzed learners and datasets, this study is one of the largest studies on bias mitigation yet presented in the literature.

CCS CONCEPTS

• Software and its engineering → Software creation and management; • Computing methodologies → Machine learning.

KEYWORDS

Software Fairness, Fairness Metrics, Bias Mitigation

ACM Reference Format:

Joymallya Chakraborty, Suvodeep Majumder, and Tim Menzies. 2021. Bias in Machine Learning Software: Why? How? What to Do?. In *Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '21), August 23–28, 2021, Athens, Greece.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3468264.3468537

1 INTRODUCTION

It is the ethical duty of software researchers and engineers to produce quality software that makes fair decisions, especially for high-stake software that makes important decisions about human lives.

Sadly, there are too many examples of machine learning software exhibiting unfair/biased behavior based on some protected attributes like sex, race, age, marital status. For example:

- Amazon had to scrap an automated recruiting tool as it was found to be biased against women [1].
- A widely used [illegible] software was found to be biased against dark-sk[illegible] [2].
- Google Transl[illegible] popular translation engine in the world, shows gender bias. She is an engineer, He is a nurse" is translated into Turkish and then again into English becomes "He is an engineer, She is a nurse" [3].

Prior works on managing fairness in machine learning software have tried mitigating bias by exploring a very wide space of control parameters for machine learners. For example, Johnson et al. [4] executed some (very long) [illegible] that looped over the set of control parameters of a [illegible] settings that reduced bias. Another approach, p[illegible] 20 by Chakraborty et al. [5] used stochastic sam[illegible] the space of control options (using some incremental feedback operator to adjust where to search next). While these approaches were certainly useful in the domains tested by Johnson and Chakraborty et al., these methods are "dumb" in a way because they do not take advantage of domain knowledge. This is a problem since, as shown by this paper, such domain knowledge can lead to a more direct (and faster and more effective) bias mitigation strategy.

The insight we offe[illegible] root causes of bias might be the *prior decisions* [illegible] *training data*. Those prior decisions affect (a) wh[illegible]ected and (b) the labels assigned to those examples. [illegible] bias, it might suffice to apply mutators to the training data in order to

> *(A) remove biased labels; and (B) rebalance internal distributions such that they are equal based on class and sensitive attributes.*

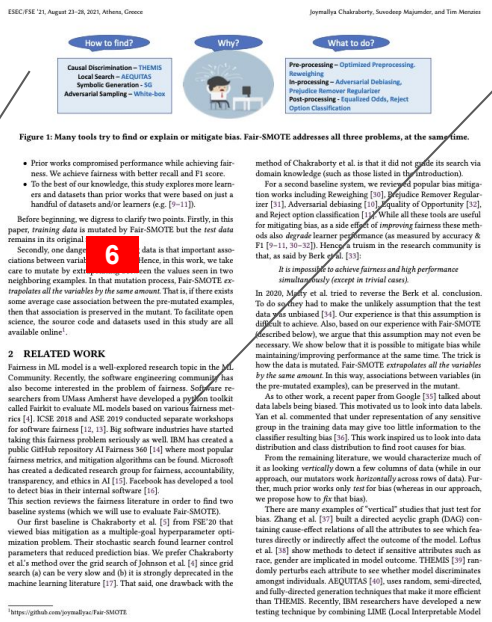
This "Fair-SMOTE" tactic uses *situation testing* [6, 7] to find biased labels. Also, it balances frequencies of sensitive attributes and class labels (whereas the older SMOTE algorithm [8] just balances the class labels). We recommend Fair-SMOTE since:

- Fair-SMOTE is *faster* (229%) than prior methods such as Chakraborty et al.'s [5] method.
- Models generated via Fair-SMOTE are more *effective* (measured in terms of recall and F1) than Chakraborty et al. [5] and another state-of-the-art bias mitigation algorithm [9].

Overall, this paper makes the following contributions:

- We demonstrate two main reasons for the training data being biased - a) data imba[illegible] biased proxy data label.
- We combine finding [illegible] bias, and removing bias.
- We show that traditi[illegible] mitigation techniques damage the fairness of the m[illegible]

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ESEC/FSE '21, August 23–28, 2021, Athens, Greece © Association for Computing Machinery: [illegible] 978-1-4503-8562-6/21/08…$15.00 https://doi.org/10.1145/3468264.3468537



ESEC/FSE '21, August 23–28, 2021, Athens, Greece Joymallya Chakraborty, Suvodeep Majumder, and Tim Menzies

Figure 1: Many tools try to find or explain or mitigate bias. Fair-SMOTE addresses all three problems, at the same time.

- Prior works compromised performance while achieving fairness. We achieve fairness with better recall and F1 score.
- To the best of our knowledge, this study explores more learners and datasets than prior works that were based on just a handful of datasets and/or learners (e.g. [9–11]).

Before beginning, we digress to clarify two points. Firstly, in this paper, *training data* is mutated by Fair-SMOTE but the *test data* remains in its original [illegible].

Secondly, one danger [illegible] data is that important associations between variab[illegible]dence, in this work, we take care to mutate by extr[illegible]en the values seen in two neighboring examples. In that mutation process, Fair-SMOTE *extrapolates all the variables by the same amount*. That is, if there exists some average case association between the pre-mutated examples, then that association is preserved in the mutant. To facilitate open science, the source code and datasets used in this study are all available online[1].

2 RELATED WORK

Fairness in ML model is a well-explored research topic in the ML Community. Recently, the software engineering community has also become interested in the problem of fairness. Software researchers from UMass Amherst have developed a python toolkit called Fairkit to evaluate ML models based on various fairness metrics [4]. ICSE 2018 and ASE 2019 conducted separate workshops for software fairness [12, 13]. Big software industries have started taking this fairness problem seriously as well. IBM has created a public GitHub repository AI Fairness 360 [14] where most popular fairness metrics, and mitigation algorithms can be found. Microsoft has created a dedicated research group for fairness, accountability, transparency, and ethics in AI [15]. Facebook has developed a tool to detect bias in their internal software [16].

This section reviews the fairness literature in order to find two baseline systems (which we will use to evaluate Fair-SMOTE).

Our first baseline is Chakraborty et al. [5] from FSE'20 that viewed bias mitigation as a multi-goal hyperparameter optimization problem. Their stochastic search found learner control parameters that reduced prediction bias. We prefer Chakraborty et al.'s method over the grid search of Johnson et al. [4] since grid search (a) can be very slow and (b) it is strongly deprecated in the machine learning literature [17]. That said, one drawback with the method of Chakraborty et al. is that it did not guide its search via domain knowledge (such as those listed in the Introduction).

For a second baseline system, we reviewed popular bias mitigation works including Reweighing [30], Prejudice Remover Regularizer [31], Adversarial debiasing [10], Equality of Opportunity [32], and Reject option classification [11]. While all these tools are useful for mitigating bias, as a side effect of *improving* fairness these methods also *degrade* learner performance (as measured by accuracy & F1 [9–11, 30–32]). Hence a truism in the research community is that, as said by Berk et al. [33]:

> *It is impossible to achieve fairness and high performance simultaneously (except in trivial cases).*

In 2020, Mary et al. tried to reverse the Berk et al. conclusion. To do so, they had to make the unlikely assumption that the test data was unbiased [34]. Our experience is that this assumption is difficult to achieve. Also, based on our experience with Fair-SMOTE (described below), we argue that this assumption may not even be necessary. We show below that it is possible to mitigate bias while maintaining/improving performance at the same time. The trick is how the data is mutated. Fair-SMOTE *extrapolates all the variables by the same amount*. In this way, associations between variables (in the pre-mutated examples), can be preserved in the mutant.

As to other work, a recent paper from Google [35] talked about data labels being biased. This motivated us to look into data labels. Yan et al. commented that under representation of any sensitive group in the training data may give too little information to the classifier resulting bias [36]. This work inspired us to look into data distribution and class distribution to find root causes for bias.

From the remaining literature, we would characterize much of it as looking *vertically* down a few columns of data (while in our approach, our mutators work *horizontally* across rows of data). Further, much prior works *only test* for bias (whereas in our approach, we propose how to *fix* that bias).

There are many examples of "vertical" studies that just test for bias. Zhang et al. [37] built a directed acyclic graph (DAG) containing cause-effect relations of all the attributes to see which features directly or indirectly affect the outcome of the model. Loftus et al. [38] show methods to detect if sensitive attributes such as race, gender are implicated in model outcome. THEMIS [39] randomly perturbs each attribute to see whether model discriminates amongst individuals. AEQUITAS [40], uses random, semi-directed, and fully-directed generation techniques that make it more efficient than THEMIS. Recently, IBM researchers have developed a new testing technique by combining LIME (Local Interpretable Model

[1] https://github.com/joymallyac/Fair-SMOTE

http: //tiny.cc/ advice22

Other standard components

1. List of attributes
2. List of attribute distributions
3. Analysis graphic
4. Lit review (takes 1..5 days)

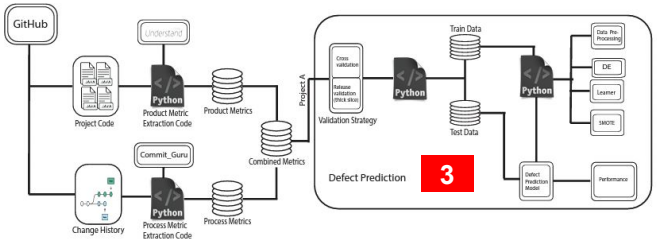


Fig. 3: Framework for this analysis.

Table 1: List of product metrics used in this study

Type	Metrics	Count
File	AvgCyclomatic, AvgCyclomaticModified, AvgCyclomaticStrict, AvgEssential, AvgLine, AvgLineBlank, AvgLineCode, AvgLineComment, CountDeclClassMethod, CountDeclClassVariable, CountDeclInstanceMethod, CountDeclInstanceVari[illegible]eclMethod, CountDeclMethodAll, CountDeclMethodDefa[illegible]eclMethodPrivate, CountDeclMethodProte[illegible]DeclMethodPublic, CountLine, CountLine[illegible]LineCode, CountLineCodeDecl, CountLineCodeExe, CountLineComment, CountSemicolon, CountStmt, CountStmtDecl, CountStmtExe, MaxCyclomatic, MaxCyclomaticModified, MaxCyclomaticStrict, MaxEssential, RatioCommentToCode, SumCyclomatic, SumCyclomaticModified, SumCyclomaticStrict, SumEssential	37
Class	PercentLackOfCohesion, PercentLackOfCohesionModified, MaxInheritanceTree, CountClassDerived, CountClassCoupled, CountClassCoupledModified, CountClassBase	7
Method	MaxNesting	1

1

Table 2: List of process metrics used in this study

adev : Active Dev Count
age : Interval between the last and the current change
ddev : Distinct Dev Count
sctr : Distribution of modified code across each file
exp : Experience of the committer
la : Lines of code added
ld : Lines of code deleted
lt : Lines of code in a file before the change
minor : Minor Contributor Count
nadev : Neighbor's Active Dev Count
ncomm : Neighbor's Commit Count
nd : Number of Directories
nddev : Neighbor's Distinct Dev Count
ns : Number of Subsystems
nuc : Number of unique changes to the modified files
own : Owner's Contributed Lines
sexp : Developer experience on a subsystem
rexp : Recent developer experience

Product Metrics

Metric Name	Median	IQR	Metric Name	Median	IQR
AvgCyclomatic	1	1	CountLine	75.5	150
AvgCyclomaticModified	1	1	CountLineBlank	10.5	20
AvgCyclomaticStrict	1	1	CountLineCode	53	105
AvgEssential	1	0	CountLineCodeDecl	18	32
AvgLine	9	10	CountLineCodeExe	29	66
AvgLineBlank	0	1	CountLineComment	5	18
AvgLineCode	7	8	CountSemicolon	24	52
AvgLineComment	0	1	CountStmt	35	72.3
CountClassBase	1	0	CountStmtDecl	15	28
CountClassCoupled	3	4	CountStmtExe	19	43.8
CountClassCoupledModified	3	4	MaxCyclomatic	3	4
CountClassDerived	0	0	MaxCyclomaticModified	2	4
CountDeclClassMethod	0	0	MaxCyclomaticStrict	3	5
CountDeclClassVariable	0	1	MaxEssential	1	0
CountDeclInstanceMethod	4	7.5	MaxInheritanceTree	2	1
CountDeclInstanceVariable	1	4	MaxNesting	1	2
CountDeclMethod	5	9	%LackOfCohesion	33	71
CountDeclMethodAll	7	12.5	%LackOfCohesionModified	19	62
CountDeclMethodDefault	0	0	RatioCommentToCode	0.1	0.2
CountDeclMethodPrivate	0	1	SumCyclomatic	8	17
CountDeclMethodProtected	0	0	SumCyclomaticModified	8	17
CountDeclMethodPublic	3	6	SumCyclomaticStrict	9	18
SumEssential	6	11			

Process Metrics

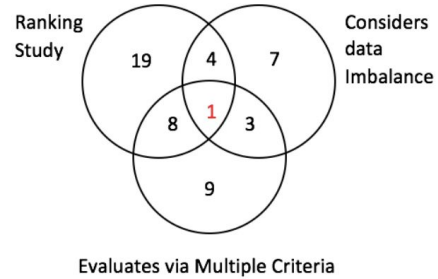
Metric Name	Median	IQR
la	14	38.9
ld	7.9	12.2
lt	92	121.8
age	28.8	35.1
ddev	2.4	1.2
nuc	5.8	2.7
own	0.9	0.1
minor	0.2	0.4
ndev	22.6	22.1
ncomm	71.1	49.5
adev	6.1	2.9
nadev	71.1	49.5
avg_nddev	2	1.8
avg_nadev	7	5.2
avg_ncomm	7	5.2
ns	1	0
exp	348.8	172.7
sexp	145.7	70
rexp	2.5	3.4
nd	1	0
sctr	-0.2	0.1

Data Statistics

Data Property	Median	IQR
Defect Ratio	37.60%	20.60%
Lines of Code	82K	200K
Number of Files	171	358
Number of Developers	31	34
Number of PRs.	55	101
Number of Commits	217	379
Duration	186(W)	191(W)
Number of Releases	20	32
Number of Defective Commits	77	139
Number of Issues	46	67
Number of unique PR submitter	5	6

2

Table 4: 22 highly cited Software Defect prediction studies.

Ref	Year	Citations	Ranked Classifiers?	Evaluated using multiple criteria?	Considered Data Imbalance?
[38]	2007	855	✓	2	✗
[32]	2008	607	✓	1	✗
[13]	2008	298	✓	2	✗
[40]	2010	178	✓	3	✗
[18]	2008	159	✓	1	✗
[30]	2011	153	✓	2	✗
[53]	2013	150	✓	1	✗
[25]	2008	133	✓	1	✗
[67]	2013	115	✓	1	✓
[35]	2009	92			
[33]	2012	79			
[28]	2007	73			
[49]	2007	66			
[27]	2009	62			
[29]	2010	60			
[17]	2015	53			
[26]	2008	41			
[62]	2016	31			
[61]	2015	27			
[50]	2012	23			
[16]	2016	15			
[4]	2017	0			

4